

Lab 1: Quantum Circuits

QML

1 introduction

welcome to the first lab! in this session u will get ur hands dirty building quantum circuits from scratch using qiskit. the goal is to get comfortable with the basic workflow: create a circuit, add gates, simulate it, and look at the results.

by the end of this lab u should be able to:

- create and manipulate quantum circuits in qiskit
- build entangled states (bell states, GHZ states)
- implement quantum teleportation
- measure qubits in different bases
- use the statevector simulator and the qasm simulator

Key Point. if u haven't installed qiskit yet, run `pip install qiskit` in ur terminal. we r using qiskit 1.x for this course. make sure u also have `qiskit-aer` installed for the simulators.

2 qiskit basics

2.1 the workflow

every qiskit program follows the same basic pattern. here's wt u need to do:

1. import `QuantumCircuit` from `qiskit`
2. create a circuit object — u specify how many qubits and how many classical bits u want
3. add gates to the circuit (hadamard, CNOT, etc.)
4. choose a backend (simulator or real device)
5. transpile and run
6. grab the results

the two simulators we care about r:

- `StatevectorSimulator` — gives u the full quantum state. great for debugging bcs u can see exactly wt the state looks like. not realistic tho, bcs on a real quantum computer u can't peek at the state.
- `QasmSimulator` — simulates actual measurements. u get a dictionary of counts, just like u would from a real device.

2.2 single qubit gates

before we start building circuits, let's review the gates we'll use. u should already know these from lecture but here's a quick refresher.

Definition 2.1. the Hadamard gate H maps $|0\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $|1\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$. in matrix form:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

the Pauli gates r also important:

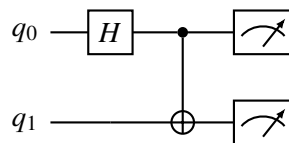
$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

and for rotation we have $R_y(\theta)$ and $R_z(\theta)$ which will be super important later when we get to variational circuits.

2.3 two qubit gates

the big one is CNOT (also called CX). it flips the target qubit if the control qubit is $|1\rangle$.

here's wt a simple circuit with H and CNOT looks like — this is the circuit that creates a bell state:



Intuition. the H gate puts q_0 into superposition, and then the CNOT entangles q_0 and q_1 . the result is $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, which is one of the four bell states. when u measure, u'll get either 00 or 11 with equal probability — never 01 or 10. that's entanglement for u.

Exercise 1. ur first circuit. create a quantum circuit with 2 qubits and 2 classical bits. apply a hadamard gate to qubit 0, then a CNOT with control on qubit 0 and target on qubit 1. add measurements to both qubits.

steps:

1. import `QuantumCircuit` from `qiskit`
2. create `QuantumCircuit(2, 2)`
3. call `.h(0)` to add hadamard on qubit 0
4. call `.cx(0, 1)` for the CNOT
5. call `.measure([0,1], [0,1])` to measure both qubits
6. draw the circuit using `.draw('mpl')`

run this on the qasm simulator with 1024 shots. plot a histogram of the results. u should see roughly 50/50 split between 00 and 11.

3 bell states

there are four bell states and they form a basis for the two-qubit space. every bell state is maximally entangled.

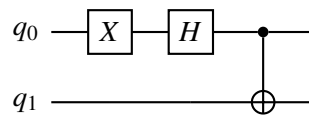
$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (1)$$

$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) \quad (2)$$

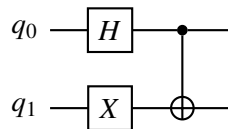
$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) \quad (3)$$

$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \quad (4)$$

the trick to building all four is to start with the right input state and then apply H + CNOT.
circuit for $|\Phi^-\rangle$:



circuit for $|\Psi^+\rangle$:



Exercise 1. all four bell states. build circuits for all four bell states. for each one:

1. construct the circuit (no measurements yet)
2. use the statevector simulator to get the output state
3. verify the state vector matches the expected bell state
4. then add measurements and run on qasm simulator with 4096 shots
5. record the measurement outcomes

hint: $|\Phi^+\rangle$ is just H + CNOT on $|00\rangle$. for $|\Phi^-\rangle$, apply X to qubit 0 first. for $|\Psi^+\rangle$, apply X to qubit 1 first. for $|\Psi^-\rangle$, apply X to both qubits first.

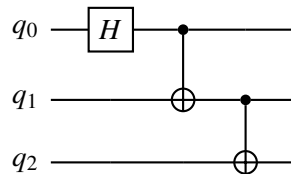
write down the mapping: which input state $\rightarrow$ which bell state?

4 GHZ states

a GHZ state is the generalization of a bell state to n qubits:

$$|\text{GHZ}_n\rangle = \frac{1}{\sqrt{2}}(|0\rangle^{\otimes n} + |1\rangle^{\otimes n})$$

for 3 qubits that's $\frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$. the circuit is pretty straightforward:



Intuition. the pattern is: H on the first qubit, then a chain of CNOTs. each CNOT spreads the entanglement to one more qubit. for n qubits u need 1 H gate and $n - 1$ CNOT gates.

Exercise 1. building GHZ states.

1. build a 3-qubit GHZ circuit. verify the statevector is $\frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$.
2. build a 5-qubit GHZ circuit. measure it and check u only get $|00000\rangle$ and $|11111\rangle$.
3. write a function that takes n as input and returns an n -qubit GHZ circuit. the function should:
 - create a `QuantumCircuit(n)`
 - apply H to qubit 0
 - loop from $i = 0$ to $n - 2$ and apply CNOT with control i and target $i + 1$
 - return the circuit
4. test ur function for $n = 2, 3, 4, 5, 8$. for each one, verify with statevector simulator.

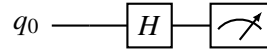
5 measurement in different bases

so far we've only measured in the computational basis (Z basis). but sometimes u want to measure in other bases, like the X basis or the Y basis.

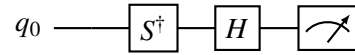
the trick: u can't directly measure in a different basis in qiskit. instead, u apply a rotation before measuring that transforms the basis u care about into the computational basis.

- X basis measurement: apply H before measuring. this works bcs H maps $|+\rangle \rightarrow |0\rangle$ and $|-\rangle \rightarrow |1\rangle$.
- Y basis measurement: apply $S^\dagger$ then H before measuring.

circuit for measuring in the X basis:



circuit for measuring in the Y basis:



Exercise 1. basis measurements.

1. prepare the state $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ by applying H to $|0\rangle$.
2. measure it in the Z basis. wt do u get? (should be 50/50)
3. measure it in the X basis (apply H before measurement). wt do u get now? (should be 100% one outcome)
4. explain why this makes sense.
5. now prepare $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ (apply X then H to $|0\rangle$). measure in the X basis. how does this differ from measuring $|+\rangle$?

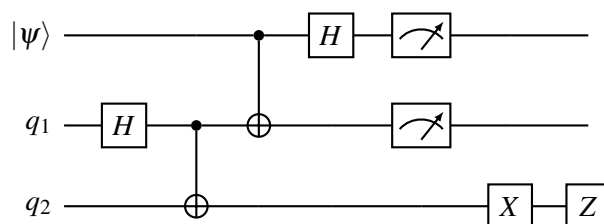
6 quantum teleportation

this is the big one for this lab. quantum teleportation lets u transfer a quantum state from one qubit to another using entanglement and classical communication.

the protocol:

1. alice and bob share a bell state $|\Phi^+\rangle$
2. alice has a qubit in some unknown state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ that she wants to send to bob
3. alice applies CNOT (her qubit controlling the bell state qubit) then H on her qubit
4. alice measures her two qubits
5. based on the measurement results, bob applies corrections (X and/or Z gates)
6. bob's qubit is now in state $|\psi\rangle$

here's the full circuit:



the first two gates (H on q_1 , CNOT on $q_1 \rightarrow q_2$) create the bell pair. then the CNOT and H on the top two qubits r alice's operations. the X and Z on the bottom qubit r bob's corrections (controlled by alice's measurement results).

Exercise 1. quantum teleportation.

1. build the teleportation circuit step by step:

- create a 3-qubit circuit with 2 classical bits (for alice's measurements)
 - prepare the state to teleport on qubit 0 — use $R_y(\pi/4)$ to make smtg interesting
 - save the statevector of qubit 0 (u'll compare later)
 - create the bell pair between qubits 1 and 2
 - apply CNOT(0,1) then H(0)
 - measure qubits 0 and 1
 - apply conditional X and Z gates on qubit 2
2. for the conditional gates, in qiskit u can use `.c_if()` or the newer `if_test` context manager. describe wt each classical bit controls.
3. verify that after teleportation, qubit 2 ends up in the same state as qubit 0 started in. u can do this by:
- running many shots and comparing measurement statistics
 - or using the statevector simulator (tho this is a bit tricky with mid-circuit measurements)
4. try teleporting different states: $|0\rangle$, $|1\rangle$, $|+\rangle$, $R_y(\pi/3)|0\rangle$.

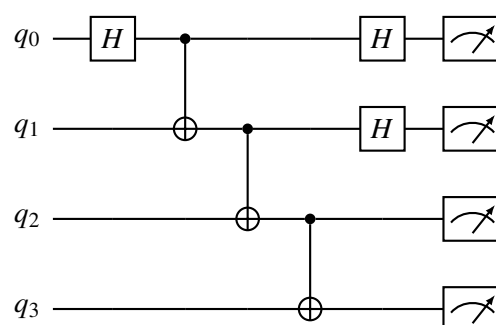
7 putting it all together

Exercise 1. circuit challenge. build a circuit that:

1. creates a 4-qubit GHZ state
2. measures qubits 0 and 1 in the X basis
3. measures qubits 2 and 3 in the Z basis
4. runs on the qasm simulator with 8192 shots

analyze the correlations between the measurement outcomes. r the X -basis results correlated with the Z -basis results? explain why or why not.

the circuit should look smtg like this:



Exercise 2. bonus: superdense coding. superdense coding is the reverse of teleportation — u send 2 classical bits using 1 qubit (plus a shared bell pair).

the protocol:

- alice and bob share $|\Phi^+\rangle$
- alice encodes her 2-bit message by applying I, X, Z, or XZ to her qubit
- alice sends her qubit to bob
- bob applies CNOT then H and measures both qubits to recover the message

implement this for all four possible messages (00, 01, 10, 11). verify bob always recovers the correct message.

8 submission

submit a jupyter notebook with:

- all exercises completed with code and output
- circuit diagrams (use `.draw('mpl')`)
- brief explanations of wt u observe
- the teleportation circuit working for at least 2 different input states